

$$x, \eta \longrightarrow \text{semantic latent } z$$

直接跳过

$$x, \eta \rightarrow image$$

这一高维中间过程。

下面详细展开这个过程。

一、首先区分“两条映射”

整个系统实际上同时存在两条路径。

真实运行路径是

$$o_t \xrightarrow{E_\phi} z_t \xrightarrow{\pi_\theta} u_t$$

其中：

$$o_t \in \mathbb{R}^{H \times W \times C}$$

是真实摄像头图像，

$$E_\phi$$

是 DINO-v2 等 semantic encoder，

$$z_t \in \mathbb{R}^{d_z}$$

是 semantic latent，

$$\pi_\theta$$

是控制器。

而用于形式化分析的是：

$$x_t, \eta_t \xrightarrow{q_\psi} \hat{z}_t \xrightarrow{\pi_\theta} u_t$$

其中：

$$x_t \in X \subseteq \mathbb{R}^n$$

是 physical state，

$$\eta_t$$

描述与视觉相关的环境变量，例如：

$$\eta = [\text{lighting, camera pose, weather, texture, occlusion, } \dots].$$

因此 q_ψ 的目标不是“随便学习一种 latent”，而是逼近：

$$q_\psi(x, \eta) \approx E_\phi(h(x, \eta))$$

这里

$$o = h(x, \eta)$$

是真实世界从 physical state 和环境条件生成图像的未知 observation process。

于是我们真正想学习的是复合映射：

$$x, \eta \xrightarrow{h} o \xrightarrow{E_\phi} z$$

但是不显式学习高维 h ，而是直接学习：

$$q_\psi \approx E_\phi \circ h.$$

这是整个思路的关键。

二、为什么这比 state-to-image 更合理

SPVT 和 SafePVC 都要学习

$$G(x, z_{\text{env}}) \approx h(x, \eta),$$

输出是完整图像：

$$G : X \times Z \rightarrow \mathbb{R}^{H \times W \times C}.$$

例如即便只是

$$128 \times 128 \times 3,$$

输出维数也有

$$49\,152.$$

但是对于控制而言，实际上绝大部分像素信息是不重要的。

假设车道保持问题的真实 physical state 是

$$x = [d, \theta, v],$$

其中：

- d : 车辆相对车道中心横向偏差；
- θ : 航向误差；
- v : 速度。

摄像头可能产生

$$640 \times 480 \times 3$$

图像，但控制器真正需要的只是类似：

lane geometry, road orientation, obstacle structure, relative spatial relations

DINO-v2 latent 已经把大量像素细节压缩成了 semantic representation。

所以不必学

$$x \rightarrow 921600 \text{ pixels},$$

而可以直接学

$$x \rightarrow 16 \sim 64 \text{ semantic variables}.$$

这会显著改善 verification scalability。

三、第一步：构造同步的 state-image 数据

首先建立数据集

$$\mathcal{D} = \{(x_i, \eta_i, o_i)\}_{i=1}^N.$$

关键是每一张图像 o_i 都必须有同步的 physical state x_i 。

在仿真环境中最容易做到。

例如 CARLA 可以同时获得：

$$x_i = [d_i, \theta_i, v_i, d_i^{obs}, \dots]$$

和摄像机画面

$$o_i.$$

并且主动改变：

$$\eta_i = [\text{weather}, \text{sun angle}, \text{camera pitch}, \text{brightness}, \text{texture}].$$

于是同一个 physical state 可以产生很多不同视觉观测：

$$(x, \eta_1) \rightarrow o_1,$$

$$(x, \eta_2) \rightarrow o_2,$$

...

$$(x, \eta_M) \rightarrow o_M.$$

这是很重要的，因为我们希望 latent model 学会区分：

state-dependent semantic information

和

visual nuisance variation.

四、真实机器人也可以获得 physical state

这并不要求系统部署时能够获得 physical state。

训练阶段可以借助：

motion capture, *GPS/RTK*, *LiDAR*, *IMU*, *simulator* *groundtruth*



得到 x_t 。

也就是说：

physical state 只是训练和验证阶段的 privileged information

部署时仍然可以完全是：

$$camera \rightarrow semanticencoder \rightarrow controller.$$

这是需要在论文里讲清楚的，否则审稿人会问：

既然你知道 x ，为什么还需要 vision controller？

答案是：

x

只在 offline training / certification 阶段可用，而 online controller 仍然只观察

o_t 。

五、第二步：使用固定 semantic encoder 得到 ground-truth latent

对于每个图像：

$$o_i,$$

通过 DINO-v2:

$$\tilde{z}_i = E_{\text{DINO}}(o_i).$$

第三篇论文正是采用 DINO-v2 作为 zero-shot encoder，利用其预训练视觉表示获得更好的空间和语义结构。 2507.13871v1

这里建议：

尽量冻结 DINO backbone

而不是从头训练整个 encoder。

原因有三个。

第一，冻结 DINO 后 latent space 比较稳定。如果不断 fine-tune encoder:

$$E_{\phi_1} \rightarrow E_{\phi_2} \rightarrow \dots$$

那么原来的：

$$q_{\psi}(x) \approx E_{\phi_1}(o)$$

会立刻失效。

第二，固定 encoder 更容易建立 conformal calibration。

第三，DINO 本身网络很大，但最终 formal verifier 不需要直接验证 DINO。

这一点非常关键。

六、第三步：不要直接使用全部 DINO feature

第三篇使用 DINO-v2-small，可得到 384 维 embedding。 2507.13871v1

但是：

维对于 formal verification 仍然偏高。

因此推荐增加一个小型 projection head:

$$P_\rho : \mathbb{R}^{384} \rightarrow \mathbb{R}^{d_z}.$$

例如:

$$384 \rightarrow 128 \rightarrow 32.$$

于是定义最终 semantic latent:

$$z = P_\rho(E_{\text{DINO}}(o))$$

其中:

$$d_z = 8, 16, 32$$

可以根据任务选择。

于是:

$$o \rightarrow 384\text{-D DINO feature} \rightarrow 16 \sim 32\text{-D safety latent}.$$

这里的 P_ρ 很重要，因为我们不是追求通用视觉表征，而是追求:

$$\text{safety-relevant semantic latent.}$$

七、什么叫 safety-relevant semantic latent

这是新方法不能含糊的地方。

普通 DINO latent 的目标是视觉语义表示，并不能保证:

$$\|z_1 - z_2\|$$

和系统安全状态的差异直接相关。

例如:

一张红色道路、一张灰色道路，DINO feature 可能差别明显;

但从控制安全角度，两张图对应的

$$d, \theta$$

可能完全一样。

反过来，两张视觉上很相似的图像可能分别对应：

$$d = 0.1m$$

和

$$d = 1.0m,$$

而后者已经接近 lane boundary。

所以需要进一步把 latent “安全化”。

可以定义一个 safety-relevant state：

$$y = c(x).$$

例如车辆：

$$y = [d, \theta, v, d_{\text{obs}}].$$

然后要求 latent 能预测这些 safety variables：

$$\hat{y} = R_{\omega}(z).$$

增加损失：

$$L_{\text{safe}} = \|R_{\omega}(z_i) - c(x_i)\|_2^2.$$

这相当于告诉 projection head：

你不需要保留所有视觉信息，但必须保留与安全相关的信息。

八、推荐的 semantic latent 学习目标

于是从图像得到 semantic latent：

$$z_i = P_{\rho}(E_{\text{DINO}}(o_i)).$$

建议训练 P_{ρ} 时使用：

$$L_{\text{semantic}} = \lambda_s L_{\text{safety}} + \lambda_t L_{\text{temporal}} + \lambda_c L_{\text{contrast}} + \lambda_r L_{\text{reg}}.$$

最重要的是第一项：

$$L_{\text{safety}} = \|R_{\omega}(z_i) - c(x_i)\|^2.$$

时间一致性可以写成：

$$L_{\text{temporal}} = \|z_{t+1} - d(z_t, u_t)\|^2.$$

这样得到的 latent 同时满足：

semantic sufficiency

和

dynamic predictability.

也就是既能表达安全信息，又适合 world model。

九、第四步：真正训练 physical-state-to-latent model

现在我们已经有了监督标签：

$$(x_i, \eta_i) \longleftrightarrow z_i.$$

于是训练：

$q_\psi(x, \eta) \rightarrow z.$

最直接的 loss 是：

$$L_{\text{map}} = \frac{1}{N} \sum_{i=1}^N \|q_\psi(x_i, \eta_i) - z_i\|_2^2.$$

其中：

$$z_i = P_\rho(E_{\text{DINO}}(o_i)).$$

所以训练链条实际上是：

$$(x_i, \eta_i, o_i)$$

先得到：

$$z_i = P_\rho(E(o_i)),$$

再训练：

$$q_\psi(x_i, \eta_i) \approx z_i.$$

因此：

q_ψ 学习的是 $E \circ h$

而不是 h 本身。

十、与 cGAN 方法的根本区别

SafePVC/SPVT 相当于：

$$x, \eta \xrightarrow{G} \hat{o} \xrightarrow{\pi} u.$$

新方法变成：

$$x, \eta \xrightarrow{q_\psi} \hat{z} \xrightarrow{\pi} u.$$

如果真实 controller 本来是：

$$o \xrightarrow{E} z \xrightarrow{\pi} u,$$

那么 verification surrogate 就是：

$$x, \eta \xrightarrow{q_\psi} z \xrightarrow{\pi} u.$$

也就是说：

$$G(x, \eta) \approx h(x, \eta)$$

被替换成：

$$q_\psi(x, \eta) \approx E(h(x, \eta)).$$

后者不需要重建：

texture, shadow, sky, roadappearance, backgroundobjects

等大量与安全证明关系不大的信息。

十一、推荐不要只用一个确定性 q_ψ

实际上，同一个 physical state：

$$x$$

在不同视觉环境下可能对应很多 latent：

$$z^{(1)}, z^{(2)}, \dots, z^{(M)}.$$

因此映射真正应该是：

$$x \rightarrow Z(x),$$

而不是：

$$x \rightarrow z.$$

最简单的建模方法是：

$$q_\psi(x) = \mu_z(x)$$

作为 nominal semantic latent。

然后定义 residual：

$$e_z = z - \mu_z(x).$$

即

$$e_z = P(E(o)) - q_\psi(x).$$

最终：

$$z = q_\psi(x) + e_z.$$

这是后续 conformal prediction 最自然的接口。

十二、state-dependent semantic uncertainty

对于每一个 calibration sample：

$$(x_i, o_i),$$

计算：

$$z_i = P(E(o_i)),$$

$$\hat{z}_i = q_\psi(x_i),$$

于是 residual：

$$e_i = z_i - \hat{z}_i.$$

如果只是取一个全局 bound：

$$\|e_i\| \leq r,$$

通常会很保守。

因为有些区域视觉非常稳定：

$$r(x) \ll 1,$$

而有些区域受到：

shadow, occlusion, motionblur, lighting

影响很大。

所以应该学习：

$$\sigma_\psi(x)$$

作为 state-dependent scale。

定义 nonconformity score：

$$r_i = \frac{\|z_i - q_\psi(x_i)\|}{\sigma_\psi(x_i) + \epsilon}.$$

然后通过 calibration set 得到 conformal quantile：

$$q_{1-\alpha}.$$

于是：

$$\mathcal{Z}_\alpha(x) = \{z : \|z - q_\psi(x)\| \leq q_{1-\alpha}\sigma_\psi(x)\}.$$

因此不再声称：

$$z = q_\psi(x)$$

精确成立，而是声称：

$$P(E(o) \in \mathcal{Z}_\alpha(x)) \geq 1 - \alpha.$$

这才是一个适合形式化安全分析的 physical-state-to-latent abstraction。

十三、如果能够显式观察环境变量 η

在仿真环境中，可以进一步学习：

$$q_\psi(x, \eta).$$

例如：

$$\eta = [I, \phi_c, \theta_c, w, r],$$

分别表示：

亮度、camera yaw、camera pitch、weather、rain。

那么 residual 会显著减小：

$$e_z = E(o) - q_\psi(x, \eta).$$

这时 conformal set：

$$\mathcal{Z}_\alpha(x, \eta)$$

会比只使用 x 时更紧。

但是在真实运行环境中，不一定能精确获得 η 。

因此更通用的方案还是：

$$q_\psi(x) + \text{state-dependent uncertainty set.}$$

十四、可以进一步做“semantic decomposition”

为了让 latent 更容易解释和验证，我更推荐把：

$$z$$

分成两部分：

$$z = [z_s, z_n].$$

其中

$$z_s$$

为 safety semantic latent,

$$z_n$$

为 nuisance latent。

希望满足：

$$z_s \approx \phi_s(x)$$

而：

$$z_n$$

主要描述：

lighting, texture, weather, background.

Controller 尽可能主要依赖：

$$z_s.$$

于是：

$$u = \pi(z_s).$$

这会非常有利于 verification。

因为验证问题从：

$$\forall(z_s, z_n)$$

变成：

$$\forall z_s \in \mathcal{Z}_s(x).$$

十五、如何训练这种解耦 latent

可以设计：

$$L = L_{\text{state}} + \lambda_{\text{inv}} L_{\text{invariant}} + \lambda_{\text{dyn}} L_{\text{dynamics}}.$$

其中：

$$L_{\text{state}} = \|R(z_s) - c(x)\|^2.$$

要求 z_s 能恢复 safety-related physical information。

然后，对于相同 x 、不同环境：

$$o_i = h(x, \eta_i), \quad o_j = h(x, \eta_j),$$

希望：

$$z_s(o_i) \approx z_s(o_j).$$

因此：

$$L_{\text{invariant}} = \|z_s(o_i) - z_s(o_j)\|^2.$$

这样会显式鼓励：

$$z_s \perp \eta \quad \text{approximately}$$

即 semantic latent 对光照、纹理等 nuisance 具有不变性。

这一步其实比普通 DINO latent 更适合安全控制。

十六、还需要保持“不同物理状态可区分”

不能只要求 nuisance invariance。

否则最极端的解就是：

$$z_s(x) = 0$$

对所有 x 。

因此还需要 state separation。

例如：

$$\|x_i - x_j\| > \delta_x$$

且安全含义不同，那么要求：

$$\|z_i - z_j\| > \delta_z.$$

可以使用 contrastive loss：

$$L_{\text{contrast}} = y_{ij} \|z_i - z_j\|^2 + (1 - y_{ij}) \max(0, m - \|z_i - z_j\|)^2.$$

最终希望 latent 满足：

$$\text{same physical semantics} \Rightarrow \text{close latent}$$

以及

$$\text{different safety semantics} \Rightarrow \text{separated latent.}$$

十七、physical state 到 latent 的网络本身应该设计得容易验证

这是和普通 representation learning 最大的区别。

不建议 q_ψ 使用大型 Transformer。

可以使用一个小型 MLP:

$$x \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow z.$$

激活函数选择:

$$\text{ReLU}, \quad \tanh$$

等 verification-friendly activation。

同时做 spectral normalization:

$$\|W_l\|_2 \leq c_l.$$

这样可以得到:

$$L_q \leq \prod_l \|W_l\|_2 L_{\sigma_l}.$$

最终:

$$\|q_\psi(x_1) - q_\psi(x_2)\| \leq L_q \|x_1 - x_2\|.$$

这样才能像 SafePVC 的 Theorem 3.1 一样, 把有限状态点验证扩展到连续区域。

SafePVC 本身就专门通过 Lipschitz 正则化和 Lipschitz 常数完成这种扩展。 720_file_Paperok

十八、最终验证时实际上不需要验证 DINO-v2

这一点是整个设计的重要优势。

DINO 网络可能有几千万参数, 直接验证几乎没有意义。

我们做的是:

训练/校准阶段:

$$o \xrightarrow{\text{DINO}} z.$$

然后建立:

$$P[z \in \mathcal{Z}_\alpha(x)] \geq 1 - \alpha.$$

于是形式化 verifier 只处理:

$$\boxed{x \rightarrow \mathcal{Z}_\alpha(x) \rightarrow \pi(z) \rightarrow f(x, u) \rightarrow B(x^+)}$$

不再展开：

$$x \rightarrow image \rightarrow DINO$$

这对于 scalability 是非常大的提升。

十九、从 semantic latent 到 physical-state safety

真正的 closed-loop dynamics 写成：

$$x_{t+1} = f(x_t, \pi(z_t)),$$

其中

$$z_t \in \mathcal{Z}_\alpha(x_t)$$

以高概率成立。

那么可以定义：

$$F_\alpha(x) = \{f(x, \pi(z)) : z \in \mathcal{Z}_\alpha(x)\}.$$

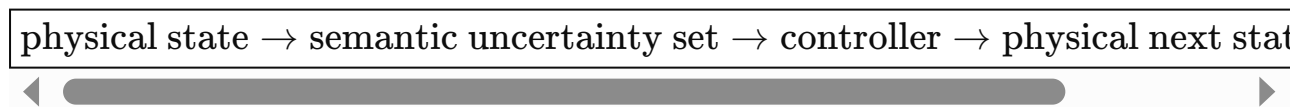
于是 verification condition 可以写成：

$$\sup_{z \in \mathcal{Z}_\alpha(x)} B(f(x, \pi(z))) \leq B(x) - \epsilon.$$

如果想使用前面提出的 stochastic barrier 形式，则为：

$$B(x) \geq (1 - \alpha) \sup_{z \in \mathcal{Z}_\alpha(x)} B(f(x, \pi(z))) + \alpha B_{\max} + \epsilon.$$

这就形成完整链：



注意最后的安全性质仍然定义在：

$$x$$

空间，而不是 latent 空间。

这是我比较推荐的版本。

二十、举一个 CARLA 的具体例子

假设：

$$x = [d, \theta, v, d_o]$$

分别是：

横向偏差、heading error、速度、前方障碍距离。

实际图像：

$$o = h(x, \eta).$$

经过 DINO：

384-D feature.

再通过 projection：

$$384 \rightarrow 32.$$

得到：

$$z \in \mathbb{R}^{32}.$$

现在对数据：

$$\{(x_i, o_i)\}_{i=1}^N$$

计算：

$$z_i = P(E(o_i)).$$

然后训练：

$$q_\psi : \mathbb{R}^4 \rightarrow \mathbb{R}^{32}.$$

例如：

$$4 \rightarrow 64 \rightarrow 64 \rightarrow 32.$$

对于：

$$x = [0.2, 0.03, 4.0, 15],$$

nominal prediction 可能是：

$$\hat{z} = q_\psi(x).$$

在晴天、阴天、黄昏、轻微遮挡下的真实 DINO latent 分别为：

$$z_1, z_2, z_3, z_4.$$

它们围绕：

$$\hat{z}$$

形成一个局部 uncertainty region：

$$\mathcal{Z}_\alpha(x).$$

Verifier 不需要知道这些图像是什么样。

它只需要证明：

$$\forall z \in \mathcal{Z}_\alpha(x),$$

控制器

$$u = \pi(z)$$

产生的下一状态都满足足够的 barrier decrease。

于是：

百万维视觉变化 $\rightarrow$ 32维 latent uncertainty

这就是这个方案真正的计算优势。

二十一、进一步建议：最终甚至可以不用直接验证 32 维 latent

还可以进一步把 uncertainty 从 latent 投影到 control space。

定义：

$$\bar{u}(x) = \pi(q_\psi(x)).$$

真实 action：

$$u = \pi(E(o)).$$

定义：

$$e_u = u - \bar{u}(x).$$

利用 calibration 直接学习：

$$\mathcal{U}_\alpha(x)$$

使得：

$$P[u \in \bar{u}(x) \oplus \mathcal{U}_\alpha(x)] \geq 1 - \alpha.$$

那么 verification 只需要：

$$\forall e_u \in \mathcal{U}_\alpha(x) :$$

$$B(f(x, \bar{u}(x) + e_u)) \leq B(x).$$

这样复杂度从：

$$d_z = 32$$

进一步降低到 action dimension：

$$d_u = 1 \sim 4.$$

因此实际上存在三个 abstraction level：

$$\boxed{\text{Image} \rightarrow \text{Semantic Latent} \rightarrow \text{Control} \rightarrow \text{Next State.}}$$

可以分别做 conformal calibration：

$$\mathcal{Z}_\alpha(x),$$

或：

$$\mathcal{U}_\alpha(x),$$

甚至直接：

$$\mathcal{E}_\alpha^x(x)$$

——即 next-state disturbance set.

二十二、对于最终论文，我推荐哪一种

最好的方案不是让 final verifier 停留在 semantic latent，而是使用两层结构：

$$\boxed{\text{semantic latent 用于建模和学习}}$$

$$\boxed{\text{physical-state disturbance 用于最终证明}}$$

即：

$$x \rightarrow q_\psi(x) \rightarrow z \rightarrow \pi(z) \rightarrow u \rightarrow x^+.$$

然后直接计算真实数据中的闭环 residual:

$$\Delta x = f(x, \pi(E(o))) - f(x, \pi(q_\psi(x)))$$

并用 conformal prediction 构造:

$$\mathcal{E}_\alpha^x(x).$$

最终系统变成:

$$x_{t+1} = f(x_t, \pi(q_\psi(x_t))) + \Delta x_t,$$

其中:

$$P[\Delta x_t \in \mathcal{E}_\alpha^x(x_t)] \geq 1 - \alpha.$$

这样从形式化验证角度最干净。

二十三、整个 physical-state $\rightarrow$ semantic-latent 流程可以浓缩为

$$(x_i, \eta_i, o_i)$$

首先:

$$o_i \xrightarrow{\text{frozen DINO}} \tilde{z}_i$$

然后:

$$\tilde{z}_i \xrightarrow{\text{safety projection}} z_i.$$

利用这些监督对:

$$(x_i, \eta_i) \leftrightarrow z_i$$

训练:

$$q_\psi(x, \eta).$$

最终:

$$z = q_\psi(x, \eta) + e_z.$$

再通过 calibration 得到:

$$P[z \in \mathcal{Z}_\alpha(x)] \geq 1 - \alpha.$$

于是整个视觉系统就从原来的：

$$x \rightarrow \text{high-dimensional image} \rightarrow \text{CNN/Transformer} \rightarrow u$$

被抽象成：

$$x \rightarrow \mathcal{Z}_\alpha(x) \rightarrow \pi \rightarrow u.$$

我认为这里真正值得作为论文创新点的，不应该只是“用了 DINO latent”，而应该定义一个更正式的概念：

Safety-Preserving Semantic Abstraction

即要求 semantic abstraction 同时满足三件事：

$$\text{Safety sufficiency} + \text{Environment invariance} + \text{Calibrated coverage}$$

其中前两个解决“latent 是否有控制意义”，第三个解决“latent approximation 是否能进入 formal guarantee”。这一步完成以后，后面的 **Conformal Stochastic Barrier Certificate** 才真正有坚实的理论基础。

  来源

ChatGPT 是 AI，可能会犯错。